

File progetto

Piano di Progetto Agile & Architetture: RehabTrack

Sistema Integrato per Centro di Riabilitazione Motoria

Candidati: Alessio Giuseppe Iacono, Salvatore Virone | Appello: 10 Luglio 2026

Docente: Prof. Ing. Luca Cruciata (A.A. 2025/2026)

1. Analisi dei Requisiti & User Stories

L'analisi dei requisiti mappa l'architettura **Three-Tier** (Presentation Tier con Ionic/Angular Standalone, Application Tier con Express in CommonJS, Data Tier con SQLite relazionale) e il controllo degli accessi basato sui ruoli (**RBAC**: *fisioterapista* e *paziente*).

EPIC 1: Autenticazione, Sicurezza & RBAC (Role-Based Access Control)

US-01 : Registrazione e Hashing Credenziali

- **User Story:** Come *Utente del sistema*, voglio *registrarmi specificando il mio ruolo clinico*, per *accedere all'area riservata di competenza*.
- **Ambito:** Shared / API
- **Priorità:** Must Have
- **Criteri di Accettazione (Gherkin):** Gherkin

```
Scenario: Registrazione con hashing della password
  Given un payload JSON contenente email, password in chiaro e role ('fisioterapista' | 'paziente')
  When viene invocato l'endpoint POST /api/auth/register
  Then il backend cifra la password con bcrypt a 10 salt rounds
  And inserisce il record nella tabella SQLite 'users'
  And restituisce lo status HTTP 201 Created escludendo l'hash dalla risposta
```

US-02 : Login Stateless e Rilascio Token JWT

- **User Story:** Come *Utente registrato*, voglio *effettuare il login con email e password*, per *ricevere un token JWT contenente il mio ruolo e l'ID univoco*.
- **Ambito:** Shared / API & Mobile/Web
- **Priorità:** Must Have

- **Criteri di Accettazione (Checklist):**

- ☐ Validazione input tramite Reactive Forms sul client (`Validators.required` , `Validators.email`).
- ☐ Verifica asincrona su SQLite tramite `User.findByEmail()` .
- ☐ Confronto crittografico tramite `bcrypt.compare(password, user.password)` .
- ☐ Firma del token tramite `jwt.sign({ id, email, role }, SECRET, { expiresIn: '1h' })` .
- ☐ Restituzione JSON con token e ruolo per il routing condizionale.

US-03 : Intercettazione Globale e Protezione Rotte Client

- **User Story:** Come *Client applicativo*, voglio *allegare automaticamente il token JWT e gestire gli errori 401*, per *garantire sessioni sicure e reindirizzamenti automatici al login*.
- **Ambito:** Mobile / Web (Angular)
- **Priorità:** Must Have
- **Criteri di Accettazione (Checklist):**
 - ☐ `httpIntInterceptor` clona la richiesta HTTP inserendo l'header `Authorization: Bearer <token>` .
 - ☐ In caso di risposta HTTP 401 Unauthorized, l'interceptor pulisce lo storage e naviga su `/login` tramite `Router.navigate()` .
 - ☐ Implementazione delle Guard funzionali (`authGuard` e `roleGuard`) per bloccare l'accesso non autorizzato ai percorsi protetti.

EPIC 2: Dashboard Clinica Fisioterapista (Desktop Web)

US-04 : Anagrafica e Registrazione Pazienti da Terapista

- **User Story:** Come *Fisioterapista*, voglio *iscrivere un paziente e associarlo alla mia anagrafica clinica*, per *poter predisporre il suo percorso riabilitativo*.
- **Ambito:** Web Desktop
- **Priorità:** Must Have
- **Criteri di Accettazione (Gherkin):** Gherkin

Scenario: Inserimento anagrafica paziente da parte del terapista

```
Given un utente autenticato con token valido e ruolo 'fisioterapista'
When compila il form con Nome, Cognome, Email e Patologia d'accesso
And invia una richiesta POST a /api/therapist/patients
Then il server crea l'utente con ruolo 'paziente' e password provvisoria
cifrata
And restituisce la lista aggiornata dei pazienti seguiti dal terapista
```

US-05 : Compositore Reattivo Schede ed Esercizi

- **User Story:** *Come Fisioterapista, voglio costruire una scheda riabilitativa personalizzata con serie, ripetizioni, tempi di recupero e vincoli posturali, per assegnare un protocollo di lavoro al paziente.*
- **Ambito:** Web Desktop
- **Priorità:** Must Have
- **Criteri di Accettazione (Checklist):**
 - ☐ Form reattivo con `FormGroup` e `FormArray` per l'inserimento dinamico di N esercizi.
 - ☐ Campi obbligatori per esercizio: `name`, `sets`, `reps_or_duration`, `rest_seconds`, `posture_notes`.
 - ☐ Salvataggio relazionale su SQLite: inserimento su tabella `cards` e inserimenti collegati su `exercises` con chiave esterna `card_id`.

US-06 : Monitoraggio Clinico, Curva del Dolore e Diario Visivo

- **User Story:** *Come Fisioterapista, voglio ispezionare i feedback sul dolore e le immagini posturali caricate dal paziente, per valutare l'efficacia del trattamento o apportare modifiche alla terapia.*
- **Ambito:** Web Desktop
- **Priorità:** Should Have
- **Criteri di Accettazione (Checklist):**
 - ☐ Chiamata `GET /api/therapist/patients/:id/logs` per recuperare cronologia sessioni, valori di dolore (1-10) e note.
 - ☐ Visualizzazione tabellare con classi di evidenziazione CSS per livelli di dolore acuto (> 7).
 - ☐ Modalità visualizzazione galleria foto posturali collegate a ciascun log.

EPIC 3: Esperienza Riabilitativa Mobile Paziente (Tabs App)

US-07 : Consultazione Scheda Giornaliera Attiva

- **User Story:** *Come Paziente, voglio visualizzare la sequenza degli esercizi prescritti per la giornata, per conoscere i dettagli della routine riabilitativa.*
- **Ambito:** Mobile (Paziente)
- **Priorità:** Must Have
- **Criteri di Accettazione (Gherkin):** Gherkin

Scenario: Recupero della scheda del giorno

Given il paziente ha effettuato il login ed è all'interno della Tab Home
When il componente invoca `GET /api/patient/today-card`

Then il backend esegue la query SQL filtrando per `patient_id`
And la vista visualizza la lista esercizi ordinata tramite la direttiva `@for` (o `*ngFor`)

US-08 : Esecuzione Guidata con Timer di Recupero sul DOM

- **User Story:** Come *Paziente*, voglio avviare un timer interattivo per scandire i tempi di riposo tra i set, per rispettare i tempi di recupero indicati dal fisioterapista.
- **Ambito:** Mobile (Paziente)
- **Priorità:** Must Have
- **Criteri di Accettazione (Checklist):**
 - ☐ Tasto di avvio pausa che attiva una funzione JavaScript asincrona basata su `setInterval()`.
 - ☐ Aggiornamento dinamico del valore numerico del conto alla rovescia sul template HTML.
 - ☐ Chiamata a `clearInterval()` allo scadere del tempo o in caso di uscita dalla pagina (`ngOnDestroy` / `ionViewWillLeave`).
 - ☐ Feedback visivo sul DOM (cambio stile CSS o badge colorato) per indicare la ripresa del set.

US-09 : Invio Report di Fine Sessione & Feedback Dolore

- **User Story:** Come *Paziente*, voglio inviare un report con la scala del dolore e note di fine sessione, per aggiornare il diario clinico del centro riabilitativo.
- **Ambito:** Mobile (Paziente)
- **Priorità:** Must Have
- **Criteri di Accettazione (Checklist):**
 - ☐ Range slider (`<ion-range min="1" max="10">`) per la selezione del livello di dolore.
 - ☐ Textarea per note descrittive dello sforzo.
 - ☐ Invio asincrono tramite `POST /api/patient/session-logs` con `card_id`, `pain_level` e `patient_notes`.
 - ☐ Persistenza su tabella `session_logs` con marcatura oraria automatica (`completed_at`).

EPIC 4: Integrazione Funzionalità Hardware & Plugin Nativi (Capacitor)

US-10 : Diario Posturale tramite Fotocamera / Galleria

- **User Story:** Come *Paziente*, voglio scattare o caricare foto posturali periodiche, per mantenere un registro visivo dell'allineamento corporeo.

- **Ambito:** Mobile (Capacitor)
- **Priorità:** Must Have
- **Criteri di Accettazione (Checklist):**
 - ☐ Invocazione di `Camera.getPhoto({ responseType: CameraResultType.Base64, source: CameraSource.Prompt, quality: 90 })`.
 - ☐ Salvataggio della stringa codificata Base64 nello storage locale tramite `@capacitor/preferences`.
 - ☐ Visualizzazione della griglia fotografica con binding `[src]="photo.webviewPath"` ed opzione di cancellazione.

US-11 : Geolocalizzazione GPS e Mappa Interattiva Clinica (Leaflet)

- **User Story:** Come *Paziente*, voglio *visualizzare la mia posizione GPS e la sede del centro medico sulla mappa*, per *calcolare la distanza chilometrica dalla struttura di riabilitazione*.
- **Ambito:** Mobile (Leaflet + GPS)
- **Priorità:** Must Have
- **Criteri di Accettazione (Checklist):**
 - ☐ Inizializzazione della mappa Leaflet (`L.map`) con layer OpenStreetMap.
 - ☐ Rilevamento coordinate GPS tramite `@capacitor/geolocation` con fallback a coordinate browser.
 - ☐ Rendering di due marker: Marker Utente (dinamico) e Marker Clinica (statico).
 - ☐ Calcolo della distanza in chilometri tramite formula geodetica o metodo Leaflet `LatLng.distanceTo()` visualizzato in un popup o badge.

2. GitHub Task Breakdown (Milestones & Issues)

RehabTrack Milestones:

- Milestone 1: Data Tier & Core Authentication (Backend / Express / SQLite)
- Milestone 2: Presentation Tier Setup & Auth Interception (Ionic / Angular)
- Milestone 3: Business Logic - Gestione Clinica Desktop (Terapista)
- Milestone 4: Business Logic - Routine Esercizi & Timer Mobile (Paziente)
- Milestone 5: Hardware Integration (Camera & Geolocation Leaflet)

Milestone 1: Data Tier & Core Authentication

- TASK-101 [DB]: Creazione dello schema SQLite relazionale (`users` , `cards` , `exercises` , `session_logs`) in `backend/db/db.js` con esecuzione `db.serialize()` e seed di prova.
- TASK-102 [Model]: Sviluppo del modello `backend/models/userModel.js` con incapsulamento delle query in `Promise native ( findByEmail , findById , create , findAllPatients )`.

- TASK-103 [Controller/Auth]: Implementazione di `backend/controllers/authControllers.js` con hashing `bcrypt` e firma JWT contenente il campo `role`.
- TASK-104 [Middleware]: Realizzazione del middleware `backend/middleware/authMiddleware.js` per l'estrazione e verifica del token Bearer.
- TASK-105 [Routes/Server]: Configurazione di `backend/routes/authRoutes.js` e montaggio in `backend/server.js` con middleware `cors` e `express.json()`.

Milestone 2: Presentation Tier Setup & Auth Interception

- TASK-201 [Ionic Setup]: Scaffolding del client Ionic 8 con Angular 19 Standalone e struttura Tabs.
- TASK-202 [Service/Auth]: Implementazione di `src/app/services/auth.service.ts` per la gestione dello stato di autenticazione e memorizzazione token.
- TASK-203 [Interceptor]: Creazione di `src/app/http-int.interceptor.ts` con iniezione del token `Authorization` e gestione centralizzata del redirect su errore 401.
- TASK-204 [Guards]: Sviluppo di `auth.guard.ts` e `role.guard.ts` per il routing protetto condizionale.
- TASK-205 [UI/Login]: Creazione della pagina `login.page.ts` con Reactive Form (`FormGroup` , `Validators`) e reindirizzamento post-login basato sul ruolo.

Milestone 3: Business Logic - Gestione Clinica Desktop (Terapista)

- TASK-301 [Model/Card]: Creazione del modello dati `cardModel.js` con metodi per inserimento schede ed estrazione esercizi per paziente.
- TASK-302 [API/Therapist]: Implementazione del controller `therapistControllers.js` e delle rotte `/api/therapist/patients` e `/api/therapist/cards`.
- TASK-303 [Web UI]: Creazione del componente dashboard per il terapista con griglia responsive (`ion-grid` , layout a due colonne con Flexbox).
- TASK-304 [Web Form]: Sviluppo del form reattivo per la composizione della scheda (set, reps, recupero in secondi, note posturali).
- TASK-305 [Feedback View]: Integrazione della vista per la consultazione dello storico dei report di dolore inviati dai pazienti.

Milestone 4: Business Logic - Routine Esercizi & Timer Mobile (Paziente)

- TASK-401 [API/Patient]: Endpoint GET `/api/patient/today-card` e POST `/api/patient/session-logs` per recupero esercizi e persistenza log.
- TASK-402 [Mobile UI]: Implementazione della schermata scheda nella Tab 1 con visualizzazione sequenziale degli esercizi tramite `@for`.

- TASK-403 [DOM Timer]: Sviluppo del componente timer con funzioni `setInterval()` e `clearInterval()` per il conto alla rovescia del recupero.
- TASK-404 [Report Form]: Implementazione del form reattivo di chiusura sessione con range slider (1-10) per il dolore e invio feedback.

Milestone 5: Hardware Integration (Camera & Geolocation Leaflet)

- TASK-501 [Camera Service]: Implementazione di `camera.service.ts` con `@capacitor/camera` (prompt foto/galleria) e persistenza via `@capacitor/preferences`.
- TASK-502 [Camera Page]: Creazione della galleria fotografica per il diario posturale con eliminazione asincrona delle immagini.
- TASK-503 [Map Leaflet]: Installazione di Leaflet e configurazione del container mappa in `map.page.ts`.
- TASK-504 [GPS Integration]: Acquisizione coordinate con `@capacitor/geolocation`, calcolo della distanza chilometrica dal centro medico e rendering marker personalizzati.

3. Divisione del Lavoro Bilanciata (2 Sviluppatori)

La suddivisione assegna la responsabilità dei moduli in modo verticale (Full-Stack per sottosistema) per minimizzare conflitti di merge su Git e garantire parallelismo durante lo sviluppo.

PIANO DI LAVORO A SPRINT	
SVILUPPATORE A (Alessio)	SVILUPPATORE B (Salvatore)
• Data Tier & Schema SQLite (db) • Model & Controller Auth (JWT) • Backend API Terapista & Paziente • Dashboard Clinica Desktop Web • Compositore Schede Esercizi • Leaflet Map & GPS Geolocation	• Setup Ionic 8 Standalone Client • AuthService, Interceptor & Guards • UI Login (Reactive Forms) • Routine Esercizi Mobile & Timer • Feedback Dolore & Range Slider • Camera Service & Diario Posturale

Fase / Modulo	Responsabile Dev A (Alessio)	Responsabile Dev B (Salvatore)
Fase 1: Backend & DB	TASK-101 (Schema DB), TASK-102 (UserModel), TASK-103 (AuthController), TASK-104 (AuthMiddleware), TASK-105 (Server setup).	Code Review, Validazione query SQL, Test chiamate API con endpoint di verifica.

Fase / Modulo	Responsabile Dev A (Alessio)	Responsabile Dev B (Salvatore)
Fase 2: Frontend & Routing	Supporto su endpoint e contratti payload JSON.	TASK-201 (Ionic Setup), TASK-202 (AuthService), TASK-203 (Interceptor), TASK-204 (Guards), TASK-205 (LoginPage).
Fase 3: Core Business Logic	TASK-301 (CardModel), TASK-302 (Therapist API), TASK-303 (Dashboard Desktop), TASK-304 (Compositore Schede), TASK-305 (Feedback View).	TASK-401 (Patient API integration), TASK-402 (UI Scheda Mobile), TASK-403 (Timer DOM setInterval), TASK-404 (Report Dolore Form).
Fase 4: Moduli Hardware	TASK-503 (Setup Leaflet), TASK-504 (GPS Geolocation & Calcolo Distanza Studio).	TASK-501 (Camera Service Capacitor), TASK-502 (UI Diario Posturale & Preferences).
Fase 5: Testing & Rilascio	Validazione integrità transazionale DB, gestione errori HTTP e vincoli referenziali.	Verifica reattività UI, test navigazione mobile su dispositivi e documentazione.

Questo piano formalizza l'architettura e la suddivisione operativa del progetto per l'appello d'esame.